

TP 6

Intégration numérique

ZFAI

Exercice 1 (Implémentation des méthodes composites). Dans cet exercice on va approcher des intégrales sur un segment $[a, b]$ en découpant ce segment de façon régulière.

1. Dans le programme suivant, on intègre une fonction f avec la méthode des rectangles à gauche

```
import math as np

def f(x) :
    return np.cos(x)

def integraleG(f,a,b,N) :
    h=(b-a)/N
    t=[a+i*h for i in range(N)]
    IN=0.
    for i in range(N) :
        IN+=h*f(t[i])
    return IN
```

Vérifier que la méthode est bien d'ordre 1 en intégrant la fonction $\cos$ entre 0 et $\frac{\pi}{2}$ (c'est-à-dire $\text{np.pi}/2$ en Python) avec différentes valeurs de N .

2. Modifier la méthode précédente pour obtenir `integraleT(f,a,b,N)` la méthode des trapèzes. Vérifier qu'elle est d'ordre 2.
3. Écrire la fonction `integraleM(f,a,b,N)` utilisant la méthode du point milieu. Comparer le résultat avec celui donné par `integraleT(f,a,b,N)`.
4. Écrire la fonction `integraleS(f,a,b,N)` utilisant la méthode de Simpson. Vérifier qu'elle est bien d'ordre quatre.
5. Utiliser la fonction `integraleS` pour intégrer la fonction $x \mapsto \sqrt{x}$ entre 0 et 1. Quelle est la vitesse de convergence en fonction de N ? Sauriez-vous en donner une explication ?

Correction

1. `1-integraleG(f,0.,np.pi/2,100)`
2.

```
def integraleT(f,a,b,N) :
    h=(b-a)/N
    t=[a+(i+1)*h for i in range(N-1)]
    IN=h/2*(f(a)+f(b))
    for ai in t :
```

```

        IN+=h*f(ai)
    return IN
3. def integraleM(f,a,b,N) :
    h=(b-a)/N
    t=[a+i*h for i in range(N)]
    IN=0.
    for i in range(N) :
        IN+=h*f(t[i]+h/2)
    return IN
4. def integraleS(f,a,b,N) :
    h=(b-a)/N
    t=[a+i*h for i in range(1,N)]
    IN=h/6*(f(a)+f(b))+2*h/3*f(a+h/2)
    for ai in t :
        IN+=h/3*f(ai)+2*h/3*f(ai+h/2)
    return IN
5. def f(x) :
    return x**0.5

```

2/3-integraleS(f,0.,1,10000)

On obtient une convergence en $O(N^{-3/2})$ ce qui est beaucoup plus lent que la vitesse attendue en $O(N^{-4})$. Cela est dû au fait que la fonction $x \mapsto \sqrt{x}$ n'est pas dérivable en 0.

Exercice 2 (Intégrale exacte de polynômes). Dans cet exercice on va intégrer des polynômes sur des segments $[a, b]$.

1. Écrire une fonction `primitivePoly(P)` qui étant donné un polynôme P représenté sous forme de liste renvoie une de ses primitives sous forme de liste aussi.
2. Écrire une fonction `integrerPoly(P,a,b)` qui renvoie la valeur exacte de $\int_a^b P(x) dx$.

Correction

1.

```
def primitivPoly(P):
    n=len(P)
    return [0]+[P[i]/(i+1) for i in range(n)]
```
2. En utilisant le TP de la semaine passée, on a la fonction :

```
def evaluation(P,a):
    n=len(P)
    S=0
    ai=1
    for i in range(n):
        S+=P[i]*ai
        ai=a*ai
    return S
```

qui permet de calculer l'intégrale comme différence de deux évaluations d'un polynôme.

```
def integrerPoly(P,a,b):
    IP=primitivPoly(P)
    return evaluation(IP,b)-evaluation(IP,a)

```

Exercice 3 (Polynômes de Legendre). Calcul des Polynômes de Legendre.

À partir de la formule de récurrence $(n+1)P_{n+1} = (2n+1)XP_n - nP_{n-1}$ et de $P_0 = 1$ et $P_1 = X$ écrire une fonction `polyLegendre(n)` qui étant donné un entier n renvoie le polynôme de Legendre P_n . Donner P_6 .

Correction

De même ici on se sert de

```
def somme(P,Q):
    l=min(len(P),len(Q))
    m=max(len(P),len(Q))
    S=[0]*m
    for i in range(l):
        S[i]=P[i]+Q[i]
    if len(P)>l:
        S[l:]=P[l:]
    elif len(Q)>l:
        S[l:]=Q[l:]
    return S
```

pour écrire la fonction

```
def polyLegendre(n):
    if n==0:
        return [1]
    if n==1:
        return [0,1]
    else:
        P0=[1]
        P1=[0,1]
        for k in range(2,n+1):
            T1=[0]+[(2*k-1)/k*P1[i] for i in range(k)]
            T2=[-(k-1)/k*P0[i] for i in range(k-1)]
            P0,P1 = P1,somme(T1,T2)
        return P1
```

Exercice 4 (Polynômes de Laguerre). Dans cet exercice on va intégrer des fonctions sur $[0, +\infty[$.

1. ★ Montrer par récurrence sur $n \in \mathbb{N}$ l'égalité $\int_0^{+\infty} x^n e^{-x} dx = n!$
2. Écrire une fonction `integrepoly(P)` qui étant donné un polynôme P représenté par une liste donne la valeur de $\int_0^{+\infty} P(x)e^{-x} dx$.
3. Écrire un programme `produitscalaire(P,Q)` qui étant donnés deux polynômes P et Q calcule le produit scalaire $\langle P, Q \rangle = \int_0^{+\infty} P(x)Q(x)e^{-x} dx$.
4. Calculer à la main les trois premiers polynômes orthogonaux pour ce produit scalaire, de coefficient dominant égal à 1. Vérifier votre calcul avec `produitscalaire`.
5. Écrire une fonction `integreR3X(f)` qui prend en argument une fonction f et qui donne une approximation de l'intégrale $\int_0^{+\infty} f(x)e^{-x} dx$ exacte pour tous les polynômes de degré au plus trois.
6. Appliquer cette fonction à $t \mapsto t^3$, à $t \mapsto t^4$ et à $t \mapsto e^{t/2}$ et comparer avec le résultat exact.

Correction

1. Pour $n = 0$ on a bien $\int_0^{+\infty} e^{-x} dx = [-e^{-x}]_0^{+\infty} = 1 = 0!$.
Supposons l'égalité vraie pour $n \in \mathbb{N}$, alors dans

$$\int_0^{+\infty} x^{n+1} e^{-x} dx$$

on fait une intégration par parties avec

$$\begin{aligned} u'(x) &= e^{-x} & v(x) &= x^{n+1} \\ u(x) &= -e^{-x} & v'(x) &= (n+1)x^n \end{aligned}$$

et on obtient

$$\int_0^{+\infty} x^{n+1} e^{-x} dx = [-e^{-x} x^{n+1}]_0^{+\infty} + (n+1) \int_0^{+\infty} x^n e^{-x} dx = (n+1) \cdot n! = (n+1)!$$

On conclut par récurrence.

- ```
2. def integrepoly(P):
 n=len(P)
 S=0.
 facn=1
 for i in range(n):
 S+=facn*P[i]
 facn=(i+1)*facn
 return S

3. def produit(P,Q):
 l=len(P)
 m=len(Q)
 S=[0]*(l+m-1)
 for n in range(l+m-1):
 for k in range(max(0,n-m+1),min(n,l-1)+1):
 S[n]+=P[k]*Q[n-k]
 return S

def produitscalaire(P,Q):
 prod=produit(P,Q)
 return integrepoly(prod)

4. On les polynômes $P_0(x) = 1$ et $P_1(x) = x + a$ pour un $a \in \mathbb{R}_-$ à déterminer
```

$$\int_0^{+\infty} P_1(x) e^{-x} dx = 1 + a$$

Donc  $a = -1$  et  $P_1(x) = x - 1$ .

Pour  $P_2(x) = x^2 + ax + b$  on a

$$\int_0^{+\infty} P_2(x) e^{-x} dx = 2 + a + b = 0 \quad \text{et} \quad \int_0^{+\infty} P_2(x) x e^{-x} dx = 6 + 2a + b = 0.$$

On résoud et on trouve  $P_2(x) = x^2 - 4x + 2$ .

P0=[1]

P1=[-1,1]

P2=[2,-4,1]

produitscalaire(P0,P1)

produitscalaire(P0,P2)

produitscalaire(P1,P2)

5. Les racines de  $P_2(x) = x^2 - 4x + 2$  sont  $x_1 = 2 - \sqrt{2}$  et  $x_2 = 2 + \sqrt{2}$ . Les polynômes interpolateurs de Lagrange correspondant à ces points sont

$$L_1(x) = \frac{x - x_2}{x_1 - x_2} = -\frac{x - (2 + \sqrt{2})}{2\sqrt{2}}$$

qui donne le poids

$$A_1 = \int_0^{+\infty} L_1(x)e^{-x} dx = \frac{2 + \sqrt{2}}{4}$$

et

$$L_2(x) = \frac{x - x_1}{x_2 - x_1} = \frac{x - (2 - \sqrt{2})}{2\sqrt{2}}$$

qui donne le poids

$$A_2 = \int_0^{+\infty} L_2(x)e^{-x} dx = \frac{2 - \sqrt{2}}{4}.$$

D'où la méthode d'intégration

$$I_2(f) = \frac{2 + \sqrt{2}}{4}f(2 - \sqrt{2}) + \frac{2 - \sqrt{2}}{4}f(2 + \sqrt{2}).$$

```
def integraleR3X(f):
 a=2**0.5
 return (2+a)/4*f(2-a)+(2-a)/4*f(2+a)
```

6. 

```
def factorielle(n) :
 if n>0 :
 return n*factorielle(n-1)
 else :
 return 1
```

```
def f(x):
 return x**3
factorielle(3)-integraleR3X(f)
```

```
def f(x):
 x**4
factorielle(4)-integraleR3X(f)
```

```
def f(x):
 return np.exp(x/2)
2-integraleR3X(f)
```

**Pour aller plus loin**

**Problème 1** (Calcul des méthodes de Gauss). On va écrire un programme qui donne les points de Gauss et les poids de Gauss de façon automatique.

1. Écrire une fonction `devP(xi)` qui étant donnée la liste des racines simples d'un polynôme  $P$  donne le polynôme  $P$  sous forme de liste de ses coefficients, et rend `P=1` si `xi` est vide.

2. Étant donnée une liste de noeuds  $(x_i)_{1 \leq i \leq n}$  pour une méthode d'intégration sur  $[-1, 1]$  pour approcher  $\int_{-1}^1 f(t) dt$ , écrire une fonction `poids(xi)` qui donne la liste des poids  $(A_i)$  correspondante pour que la méthode d'intégration soit exacte pour tous les polynômes de degré au plus  $n$ .
3. Étant donné un polynôme  $P \in \mathbb{R}[X]$  et un intervalle  $[a, b]$ , écrire une fonction `racine(P, a, b)` qui, si  $P(a)P(b) < 0$ , renvoie une racine de  $P$  dans  $[a, b]$  avec une précision relative telle que  $|P(r)| < 10^{-15} \max_i(P_i)$  où  $P_i$  est la liste des coefficients de  $P$ . On pourra utiliser la méthode de Newton et faire une dichotomie en cas d'échec.
4. Soit un polynôme  $P$  avec  $N$  racines sur  $[-1, 1]$ , on pose  $h = \frac{1}{N^2}$  et  $a_k = -1 + kh$  pour  $0 \leq k \leq 2N^2$ . Écrire une fonction `toutesracines(P)` qui pour chaque intervalle  $[a_k, a_{k+1}]$  va vérifier si  $P(a_k)P(a_{k+1}) < 0$  et dans la cas positif calculer cette racine, et au final renvoyer la liste de toutes les racines de  $P$ .
5. Grâce au résultat de l'exercice 3, écrire la fonction `Gauss(N)` qui donne les points et les poids de la méthode de Gauss à  $N$  points.
6. Implémenter la méthode composite `integraleGauss6(f, a, b, N)` à six points en découpant l'intervalle  $[a, b]$  en  $N$  sous-intervalles, tester sa précision sur  $\int_0^1 t^{12} dt$  et donner son ordre.

### Correction

```

1. def devP(xi):
 P=[1]
 for x in xi:
 P=produit(P, [-x,1])
 return P

2. def poids(xi):
 Ai=[]
 for x in xi:
 temp=xi.copy()
 temp.remove(x)
 Li=devP(temp)
 I=integrerPoly(Li,-1,1)
 c=1.
 for y in temp:
 c=c*(x-y)
 Ai.append(I/c)
 return Ai

3. def derive(P):
 n=len(P)
 return [i*P[i] for i in range(1,n)]

def racine(P,a,b):
 if evaluation(P,a)*evaluation(P,b)<0:
 dP=derive(P)
 r=(a+b)/2
 M=max([abs(p) for p in P])
 for i in range(6):
 r=r-evaluation(P,r)/evaluation(dP,r)
 if r>a and r<b and abs(evaluation(P,r))<10**(-15)*M:
 return r

```

```

 else:
 c=(a+b)/2
 if evaluation(P,a)*evaluation(P,c)>0:
 return racine(P,c,b)
 elif evaluation(P,c)==0.:
 return c
 else:
 return racine(P,a,c)
4. def toutesracines(P):
 xi=[]
 N=len(P)-1
 h=1/N/N
 ak=[-1+k*h for k in range(2*N*N+1)]
 for k in range(2*N*N):
 if evaluation(P,ak[k])==0.:
 xi.append(ak[k])
 if evaluation(P,ak[k])*evaluation(P,ak[k+1])<0:
 xi.append(racine(P,ak[k],ak[k+1]))
 return xi
5. def Gauss(N):
 P=polyLegendre(N)
 xi=toutesracines(P)
 Ai=poids(xi)
 return xi,Ai
6. x6,A6 = Gauss(6)

```

```

def integraleGauss(f,a,b,N):
 h=(b-a)/N
 t=[a+i*h for i in range(N)]
 IN=0.
 for i in range(N) :
 for k in range(6):
 IN+=h*A6[k]/2*f(t[i]+h*(x6[k]+1)/2)
 return IN

```

```

def f(x) :
 return x**12
1/13-integraleGauss(f,0,1,4)

```

L'ordre théorique est 12 car cette méthode est exacte pour les polynômes de degré 11. Et c'est bien ce qu'on observe avec cet exemple où l'erreur est en  $O(h^{-12})$  : elle est divisée par 4000 lorsque  $h$  est divisé par 2.